DP-OPS-0021 Post-Implementation Audit and Strategic Architecture Report: The Consolidation of the Project Factory

1. Executive Strategic Analysis

The Stela platform, a high-governance repository designed for explainability and provenance-forward software manufacturing, has recently executed a critical transition codified in Dispatch Packet DP-OPS-0021 ("The Autopoietic Project Binary"). This transition marks a fundamental ontological shift from a static repository structure to a dynamic "Project Factory"—a system capable of manufacturing, maintaining, and validating compliant software payloads through high-density governance and automated context injection. The successful execution of this phase is not merely a tooling update; it represents the maturation of the system into a state of "Partial Autopoiesis," where the repository possesses the internal mechanisms to maintain its own structural integrity and reproduce compliant project environments.[1]

This comprehensive research report serves three primary strategic functions. First, it provides an exhaustive forensic audit of the implementation of DP-OPS-0021, verifying that the objectives outlined in the "Stela Agent Constitution Refinement" document have been met with sufficient fidelity to the system's core axiom: the "Economy of Truth".[1] Second, it investigates and resolves the "Red Gate" anomaly currently affecting the repository's Continuous Integration (CI) pipeline, tracing the root cause to specific path-resolution vulnerabilities within the tools/ directory scripts and proposing a robust, git-anchored solution.[2] Third, and most significantly, it outlines the architectural roadmap for the "Strengthening of Projects"—a concerted effort to integrate Agents, Tasks, and Skills into a unified, modular execution engine, thereby rendering legacy documentation artifacts obsolete and resolving the ontological tension between the System Constitution and the Project Payload.

The analysis confirms that while the core objectives of DP-OPS-0021—specifically the refactoring of ops/bin/llms and ops/bin/open, and the instantiation of ops/bin/project—have been achieved, the system requires immediate hardening of its "connective tissue." The persistence of artifacts such as ops/lib/project/INTEGRATION.md and user-STELA.md represents "Epistemic Debt"—redundant truth sources that dilute the authority of the executable canon. Furthermore, the "Repo-Gates" failure highlights a fragility in the system's self-verification logic that must be addressed to restore the "Green Gates Only" non-negotiable standard defined in docs/GOVERNANCE.md.[1] This document asserts that to achieve the next level of governance, Stela must rigorously enforce the "Single Source of Truth" (SSOT) axiom by converting static documentation into executable logic and strictly

segregating "System Constitution" from "Project Resources."

2. Theoretical Framework: The Epistemology of High-Governance

To rigorously evaluate the success of DP-OPS-0021 and define the requirements for the next phase, it is necessary to deconstruct the epistemic mechanics that govern the Stela repository. The governance of an AI-driven repository is not a matter of file management; it is a problem of the "Political Economy of Truth." In an agentic environment, truth is a finite resource quantified in tokens, and the efficiency of the system is determined by the "Attention Voltage"—the capacity of the model to attend to relevant instructions amidst competing noise.[1]

2.1. The Economy of Truth vs. The Economy of Scarcity

The transition to High-Density Governance in Phase Two was predicated on the "Economy of Truth"—the axiom that for a proposition to be valid, the resources required to derive it must be explicitly present.[1] This drove the "Great Compression" of TRUTH.md and AGENTS.md. However, forensic analysis suggests a divergence between "Economy" (optimization) and "Scarcity" (deprivation). Epistemic Scarcity occurs when unresolvable unknowns prevent probabilistic optimization. In Stela, this manifests when the removal of narrative justification strips away the semantic causal links required for the model to bridge the gap between "Instruction" and "Intent."

The correction implemented in DP-OPS-0021, and validated in this report, is the refinement of axioms to be declaratively absolute. The instruction must be self-verifying. The shift from "social contract" language to "technical specification" language in CONTRIBUTORY.md represents this shift. This aligns with recent research in Agentic Software Engineering (SE 3.0), which emphasizes that while human-centric rules are inverted, foundational principles like Modularity and Separation of Concerns become even more critical for autonomous agents.[4]

2.2. Autopoiesis and the Project Factory

The ultimate goal of Stela is Autopoiesis—the capability of the system to reproduce and maintain itself. A system is autopoietic if its maintenance processes are internal to its operation. The ops/bin/project utility is the primary engine of this autopoiesis. It acts as a "Membrane" that selectively permeates context between the Platform (the OS) and the Payload (the Project). This separation is vital because the code in projects/ has a different lifecycle, ownership, and truth table than the code in ops/. Indiscriminately loading "Project Agents" into the "System Context" creates "Context Pollution," risking a collapse into a "Vector Haze" where the model cannot distinguish between immutable system laws and mutable

project requirements.[1]

Concept	Definition in Stela Context	Implementation Artifact
Autopoiesis	Self-creation and self-maintenance.	ops/bin/project, ops/bin/prune
Membrane	The interface separating System from Payload.	ops/bin/project work
Epistemic Density	High information value per token.	AGENTS.md, TRUTH.md
Context Hygiene	Prevention of irrelevant information loading.	ops/lib/manifests/CONTEXT.md

3. Forensic Audit of DP-OPS-0021 Implementation

A granular, line-by-line forensic verification of the repository state (Freshness: f2258f404) against the requirements of DP-OPS-0021 reveals a highly successful implementation, though specific areas require optimization to fully meet the "High-Governance" standard.

3.1. The Project Binary Interface (ops/bin/project)

Requirement: The placeholder file for ops/bin/project must be replaced with a robust wrapper script that serves as the "Factory Floor Manager," capable of scaffolding projects and managing their lifecycle to eliminate the "Ghost in the Machine" phenomenon.[1]

Audit Findings: The file ops/bin/project is no longer a placeholder. It is a fully executable Bash script that sources logic from ops/lib/scripts/project.sh.[1] The script implements three distinct subcommands that align with the autopoietic requirement:

1. **init**: This command successfully scaffolds new projects using the template at ops/lib/project/SCAFFOLD.md. It correctly registers the new entity in the ops/lib/manifests/PROJECTS.md registry, ensuring that no project exists as an "orphan" outside the system's awareness. It supports metadata injection (Display Name, Notes) and includes a --dry-run safety mechanism, which is a critical feature for high-governance operations where state changes must be predictable.
2. **work**: This is the critical "Membrane" function. It wraps the ops/bin/open command and dynamically injects project-specific context—specifically projects/<ID>/STELA.md and

projects/<ID>/README.md—into the session. This verifies the successful creation of the "Project Binary Paradigm," where project context is loaded Just-In-Time (JIT) rather than polluting the global namespace. The script enforces the precedence of TRUTH.md by loading it alongside the project context, preventing the project from overriding system invariants.

3. **status**: This command provides a tabular view of registered projects, ensuring visibility into the "Factory Floor." It reads directly from the SSOT manifest, reinforcing the data-driven nature of the operations.

Verdict: COMPLETE. The binary successfully establishes the membrane between Platform and Payload, resolving the "Ghost in the Machine" issue identified in the strategic analysis.

3.2. Dynamic Context Bundling (ops/bin/llms)

Requirement: The context generation script ops/bin/llms must move from hardcoded lists to a manifest-driven approach, supporting dynamic profiling (e.g., --profile=architect) to support "Agent Specialization" without "Context Pollution".[1]

Audit Findings:

The script ops/bin/llms has been extensively refactored to adhere to the Single Source of Truth (SSOT) principle.

- **Manifest-Driven Architecture:** The script now reads ops/lib/manifests/CONTEXT.md to generate the "Small" and "Full" bundles. It parses the markdown sections using awk, which ensures that the manifest file itself drives the behavior of the binary.[1] This eliminates the "SSOT Violation" where the binary and the manifest could drift apart.
- **Dynamic Profiling:** The script includes specific logic for the --profile flag.
  - --profile=architect: It dynamically walks the directory tree to find docs/CODEMAPS within the target project or global docs. This allows an "Architect" agent to see the structural maps without being burdened by unrelated operational details.
  - --profile=security: It dynamically loads docs/security, providing the "Security Reviewer" with the necessary policy framework.
- **Context Hygiene Enforcement:** Crucially, the script includes a validate_bundle_paths function that explicitly checks for "Context Hazards" (defined as docs/library/agents, docs/library/tasks, docs/library/skills) and aborts if they are found in the default bundles. This rigor enforces the "Economy of Truth" by preventing context pollution, ensuring that these heavy artifacts are only loaded JIT.[1]

Verdict: COMPLETE. The script effectively implements the "tiered context" requirement and adheres to the strict hygiene rules mandated by AGENTS.md.

3.3. Agent Persona Injection (ops/bin/open)

Requirement: The session initiator ops/bin/open must support an --agent flag to load

specialized personas from docs/library/agents/, replacing the generic "Contractor" definition.

Audit Findings:

The ops/bin/open script has been updated to accept --agent=name.

- **Resolution Logic:** The script scans docs/library/agents/R-AGENT-*.md, parses the YAML frontmatter to match the name field, and extracts the "Role" body content.[1] This mechanism allows the system to treat agents as modular, version-controlled software components rather than vague prompt instructions.
- **Injection Strategy:** The extracted role replaces the generic "Contractor" definition in the system prompt. This allows for the precise "Agent Specialization" mandated by Phase 3. For example, loading R-AGENT-06 (Security Reviewer) transforms the LLM's stance from a generic assistant to a specialized auditor with specific tools and constraints.[1]
- **SSOT Adherence:** By reading directly from the docs/library/agents files, the script ensures that the documentation *is* the code. There is no separate "config" file for agents; the definition file is the single source of truth.

Verdict: COMPLETE. The implementation enables the JIT loading of specialized intelligence, a prerequisite for the "Trinity Integration" discussed later in this report.

3.4. Semantic Refinement (CONTRIBUTORY.md & AGENTS.md)

Requirement: Create CONTRIBUTORY.md as a high-density instruction set and purge narrative fluff from AGENTS.md to reduce token cost and interpretation drift.[1]

Audit Findings:

- **CONTRIBUTORY.md**: This file now exists at the root. It has been stripped of all "Welcome" messages, emojis, and "cleverness" found in the human-facing CONTRIBUTING.md. It presents strictly imperative rules: "Do not push to main," "Work only on work/* branches," "Every PR must pass repo-gates".[1] This meets the definition of High-Density Governance, targeting the "Contractor" (AI) rather than the "Contributor" (Human).
- **AGENTS.md**: The specific line identified as a violation—"This ensures rhythmic precision for technical auditing"—has been purged. The file now reads: "Linguistic Precision: No contractions.".[1] The directives are Boolean and axiomatic, reducing the risk of "Interpretation Drift" where an agent might debate the meaning of "rhythmic precision."

Verdict: COMPLETE. The epistemic density of the governance layer has been successfully optimized.

---

4. Forensic Analysis of "Repo-Gates" Anomaly

The user reports that "repo-gates is red" due to "strange path reporting from scripts located in /tools." This represents a failure in the "Anti-Drift Governance" and requires immediate remediation. A deep forensic analysis of the shell scripts in tools/ reveals a subtle but critical vulnerability in how path resolution is handled, particularly in Continuous Integration (CI) environments where the working directory and git state may differ from a local developer's environment.

4.1. The Diagnosis: Relative Path Fragility

The primary linter scripts—tools/lint/truth.sh, tools/lint/context.sh, and tools/lint/library.sh—rely on establishing the REPO_ROOT to locate canonical files like TRUTH.md or ops/lib/manifests/CONTEXT.md.

In the current dump, tools/lint/context.sh uses the following logic [1]:

Bash

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"  # This only goes up one level!
MANIFEST_PATH="${REPO_ROOT}/ops/lib/manifests/CONTEXT.md"
```

The Logic Failure:

The script tools/lint/context.sh is located in tools/lint/.

1. dirname gives tools/lint.
2. SCRIPT_DIR becomes /abs/path/to/tools/lint.
3. REPO_ROOT is defined as cd ${SCRIPT_DIR}/... This resolves to /abs/path/to/tools.
4. The script then constructs the path ${REPO_ROOT}/ops/lib/manifests/CONTEXT.md.
5. This resolves to /abs/path/to/tools/ops/lib/manifests/CONTEXT.md.

This path does not exist. The ops/ directory is in the repository root, not inside tools/. The script assumes it is one level deep (tools/script.sh), but it is actually two levels deep (tools/lint/script.sh). This "Path Depth Mismatch" causes the script to fail to find the manifest, triggering the red gate.[2]

Similarly, tools/verify.sh defines:

Bash

```
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE}")/.." && pwd)"
```

Since verify.sh is in tools/ (Depth 1), going up one level (..) correctly takes it to the Repo Root. The linters in tools/lint/ (Depth 2) inherit this pattern but fail to adjust the depth count.

4.2. The Fix: Git-Anchored Root Resolution

To permanently resolve this and prevent future drift if scripts are moved, we must abandon relative path counting (../..) and rely on the definitive authority of git. In a "High-Governance" repository, *Location Independence* is a requirement. Scripts must verify their own execution context based on the "Truth" of the repository structure (git) rather than the "Reality" of the file system (pwd).[2]

Proposed Logic Pattern (The "Anchor"):

Bash

```
# Robust Repo Root Resolution
if command -v git >/dev/null 2>&1 && git rev-parse --is-inside-work-tree >/dev/null 2>&1; then
    REPO_ROOT="$(git rev-parse --show-toplevel)"
else
    # Fallback for non-git environments (e.g., rigid container mounts)
    # Assumes script is at tools/lint/script.sh (2 levels down)
    SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
    # Attempt to resolve root by looking for TRUTH.md
    if]; then
        REPO_ROOT="$(cd "${SCRIPT_DIR}/../.." && pwd)"
    elif]; then
        REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
    else
        echo "ERROR: Could not locate REPO_ROOT (TRUTH.md not found)." >&2
        exit 1
    fi
fi
```

By forcing REPO_ROOT to be determined by git rev-parse --show-toplevel, we ensure that no matter where the script lives (depth 1, 2, or 5), it always correctly identifies the base of the repository. This aligns with the "Anti-Drift Governance" directive.

4.3. Specific Refinements for tools/lint/ Scripts

1. **tools/lint/context.sh**:
   - *Current:* REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)" (Resolves to tools/).
   - *Fix:* Implement the Git Anchor. Verify ops/lib/manifests/CONTEXT.md exists relative to the discovered root.
2. **tools/lint/truth.sh**:
   - *Current:* Does not explicitly define REPO_ROOT. It relies on git ls-files working from the current working directory (CWD).
   - *Risk:* If a user runs cd tools/lint;./truth.sh, git ls-files will return different paths or fail to find ops/ files because they are "above" the CWD.[6]
   - *Refinement:* Explicitly cd "$REPO_ROOT" at the start of execution to normalize the environment before running git ls-files.
3. **tools/lint/library.sh**:
   - *Current:* REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)" (Incorrect; resolves to tools/).
   - *Fix:* Implement the Git Anchor.

This remediation will be the primary objective of the upcoming Dispatch Packet (DP-OPS-0022), as "Green Gates Only" is a non-negotiable governance rule.

---

5. The Strengthening of Projects: A Unified Library Architecture

The user has identified a critical need to "strengthen and improve" the projects system, specifically regarding the integration of Agents, Tasks, and Skills. Currently, these concepts exist as static markdown files in docs/library/. The goal is to make them "work together" dynamically within the Project Binary paradigm. This section outlines the architectural pivot from "Passive Library" to "Active Arsenal" and addresses the placement of SKILL.md.

5.1. The Library Trinity: Agents, Tasks, Skills

To streamline the repository and enforce the "Economy of Truth," we must strictly define the ontological role of each component in the library. This clarification resolves the "Ontological Tension" between the general-purpose system and the specific project needs.[1]

The proposed architecture defines a "Trinity" of execution:

Component	Directory	Function (The "Who/How/W	Metaphor	Relationship to System

		hat")		
Agent	docs/library/agents/	**WHO** performs the work. Defines the persona, toolset, and cognitive stance.	The Worker	**Citizen.** Adheres to AGENTS.md (Constitution).
Task	docs/library/tasks/	**HOW** the work is orchestrated. Defines the workflow, sequence of agents, and handoff protocols.	The Recipe	**SOP.** Instantiated into TASK.md (Active Contract).
Skill	docs/library/skills/	**WHAT** specific knowledge is applied. Defines granular, reusable procedures and anti-drift checks.	The Tool	**Capability.** Logged in SKILL.md (Registry).

Logic Flaw Resolution: Currently, AGENTS.md and docs/library/agents/ creates a confusion of authority.

- **Clarification:** AGENTS.md is the **Constitution**. It defines the *rights* and *laws* governing all agents (e.g., "No contractions," "Anti-Drift"). It does not define *specific* agents.
- **Clarification:** docs/library/agents/ contains the **Roster**. These are the specific specialized personas that obey the Constitution.

The Proposal:

The ops/bin/project work command must evolve to allow the composition of these three

elements via flags.

- *Command:* ops/bin/project work P-01 --agent=architect --task=B-TASK-03 --skill=S-LEARN-05
- *Effect:* The prompt is constructed by stacking:
  1. TRUTH.md + AGENTS.md (Constitution)
  2. docs/library/agents/R-AGENT-01.md (The Architect Persona)
  3. docs/library/tasks/B-TASK-03.md (The Integration Workflow)
  4. docs/library/skills/S-LEARN-05.md (Security Protocols)
  5. projects/P-01/README.md (The Project Context)

This dynamic stacking eliminates the need for static "Integration" documents that describe *how* to combine them. The *combination* happens at runtime via the binary, fulfilling the promise of agentic software development where modularity and separation of concerns are critical.[4]

5.2. File Deprecation and Consolidation

The user correctly identifies several files as candidates for removal. In a "High-Governance" repository, redundancy is drift. Any file that describes a process that can be codified in a binary is "Documentation Debt."

1. **ops/lib/project/user-STELA.md**: **DELETE.**
   - *Reasoning:* "User-level" configurations create hidden state. A project should be fully defined by its own STELA.md and the system canon. Personal preferences (e.g., IDE settings) pollute the reproducibility of the "Factory." If a rule is good, it belongs in the Project STELA.md or the System AGENTS.md. If it is personal, it belongs in the user's local environment (.git/info/exclude), not the shared repo.[8]
2. **ops/lib/project/INTEGRATION.md**: **DELETE.**
   - *Reasoning:* This file currently describes the *contract* for the integration tool ("Input: workflow selection... Output: final report"). Since ops/bin/project will now implement this logic via the work command composition, the text file is redundant. The code becomes the documentation.
3. **docs/library/INTEGRATION.md**: **DELETE.**
   - *Reasoning:* This is the "Operator Guide" for integration. It essentially points to the file above. If we implement the logic in the binary, the manual entry in docs/library/MANUAL.md suffices. "How to run integration" becomes simply ops/bin/project work --task=....

5.3. The "SKILL.md" Placement Doctrine

The Question: "Does SKILL.md need to still be a root level file?"

Analysis:

SKILL.md currently serves two purposes:

1. **Template:** It contains the "Promotion Packet" structure used by operators to promote drafts.
2. **Registry/Log:** It acts as a chronological ledger of "Candidates" and "Promotions".[1]

In the Stela filing doctrine (TRUTH.md):

- ops/ = Run (Automation)
- docs/ = Explain (Knowledge)
- Root = **Constitution & State**.

SKILL.md functions similarly to SoP.md. SoP.md is the "State of Play"—a high-level ledger of system evolution. SKILL.md is the "State of Capability"—a high-level ledger of *learned* evolution. However, unlike SoP.md which tracks *governance* changes, SKILL.md tracks *procedural* knowledge. It is closer to the library than the constitution.

Verdict: MOVE to docs/library/SKILLS.md.

The root should be reserved for the *Constitution* (TRUTH, AGENTS) and the *Active Contract* (TASK). SoP remains at root because it is the primary audit trail. SKILL.md is essentially a *library index extension*. Moving it to docs/library/ aligns with the "Filing Doctrine" by grouping "Explanation" (Skills) with the library, while keeping the root clean for "State" and "Law."

- *Action:* Update ops/lib/scripts/skill.sh to point SKILL_FILE to docs/library/SKILLS.md.
- *Action:* Update llms.txt to reflect the new path.

5.4. Relationship Definitions (The "Ontological Map")

To answer the user's specific request for proposals on relationships:

1. **AGENTS.md vs. docs/library/agents/**
    - **Relationship:** *Constitution vs. Citizen*.
    - **Proposal:** AGENTS.md defines the **Non-Negotiable Behavioral Constraints** (No contractions, Literalism, Anti-Drift) that apply to *all* AI entities. docs/library/agents/ defines the **Job Descriptions** (Architect, Reviewer) and their specific toolsets.
    - **Enforcement:** ops/bin/open MUST load AGENTS.md *first*, then inject the specific Agent file *second*. The Agent file cannot override AGENTS.md.
2. **TASK.md vs. docs/library/tasks/**
    - **Relationship:** *Active Contract vs. Standard Operating Procedure (SOP)*.
    - **Proposal:** TASK.md is the **Live Workspace** for the current session. It is ephemeral-to-the-goal (it changes with every DP). docs/library/tasks/ contains **Immutable Recipes** (e.g., B-TASK-01 Code Review).
    - **Usage:** An operator copies a *recipe* from docs/library/tasks/ into TASK.md to execute it. TASK.md is the instantiation of a B-TASK. This mirrors the Object-Class relationship in programming.

---

6. Execution Plan: DP-OPS-0022

Based on the audit and architectural decisions, the following Dispatch Packet (DP-OPS-0022) is proposed to finalize the "Project Factory" structure.

6.1. Refinement Checklist

1. **Script Hardening:** Update tools/lint/context.sh, tools/lint/truth.sh, and tools/lint/library.sh to use git rev-parse --show-toplevel for robust root resolution. This resolves the Red Gate.
2. **File Purge:** Delete ops/lib/project/user-STELA.md, ops/lib/project/INTEGRATION.md, and docs/library/INTEGRATION.md.
3. **Skill Relocation:** Move SKILL.md to docs/library/SKILLS.md and update ops/lib/scripts/skill.sh.
4. **Project Enhancement:** Update ops/bin/project to support --task and --skill flags in its work command, allowing it to automatically include those file paths in the generated OPEN prompt.

---

7. Deep Dive Report: System Architecture and Logic

7.1. The Autopoietic Loop

The successful implementation of ops/bin/project closes the autopoietic loop. The system can now:

1. **Sense:** ops/bin/status reads the registry.
2. **Decide:** TASK.md defines the intent.
3. **Act:** ops/bin/project work generates the environment.
4. **Remember:** ops/lib/scripts/skill.sh captures the learning.

This loop is distinct from standard CI/CD because it includes the *Agent's cognition* as a managed component. By formalizing AGENTS.md and docs/library/agents, we treat the "Prompt" not as a transient chat interaction, but as a version-controlled software artifact.[9]

7.2. Epistemic Scarcity and the "No-Narrative" Rule

The removal of the justification "This ensures rhythmic precision..." from AGENTS.md is a subtle but profound optimization. In Large Language Models (LLMs), every token competes for attention. Narrative justifications ("Why we do this") dilute the imperative force of the rule ("Do this"). By stripping the "Why," we increase the "Epistemic Density" of the "What." The model is less likely to hallucinate an exception to the rule if it is presented as an axiom rather

than an argument. This adheres to the "Economy of Truth" principle.[1]

7.3. The Trinity Integration Strategy

The future of Stela lies in the combinatorial power of the Library Trinity.

- **Scenario:** A Security Audit.
- **Old Way:** Operator manually reads docs/security/README.md, opens a chat, pastes content.
- **New Way (Project Factory):**
  Bash
  ```
  ops/bin/project work P-AUTH-MODULE \
   --agent=security-reviewer \
   --task=B-TASK-01 \
   --skill=S-LEARN-05
  ```

  This command assembles a "Hyper-Context" specifically tuned for the mission. It loads the **Security Reviewer** persona (Who), the **Code Review** checklist (How), and the **Security Protocols** knowledge base (What), all scoped to the **P-AUTH-MODULE** (Where). This is the realized vision of the "Autopoietic Project Binary."

---

8. Conclusions and Recommendations

The implementation of DP-OPS-0021 has been successful in establishing the mechanical foundation of the Project Factory. The project binary effectively serves as the membrane between System and Payload. However, the system is currently hindered by "Lint Rot" in the tools directory and "Documentation Debt" in the library.

Final Recommendations:

1. **Immediate Action:** Author and execute **DP-OPS-0022** (drafted below) to fix the tools/ pathing issues and resolve the Red Gate. This is the highest priority.
2. **Consolidation:** Execute the deletion of the identified INTEGRATION and user-STELA files. Move SKILL.md to docs/library/.
3. **Evolution:** Extend ops/bin/project to support the --task and --skill flags in its work command.

The Stela platform is no longer just a repository; it is a governable, introspective software engine. The completion of these next steps will transition it from "Experimental" to "Operational."

DRAFT DP-OPS-0022

Status: DRAFT

Intent: Fix Repo-Gates & Consolidate Project System

Ref: DP-OPS-0021 Audit Findings

3.1 Freshness Gate

Base Branch: main

Required Work Branch: work/fix-gates-consolidate-2026-02-04

3.2 Required Context Load

- TRUTH.md, AGENTS.md, SoP.md, ops/lib/manifests/CONTEXT.md
- tools/lint/context.sh, tools/lint/truth.sh, tools/lint/library.sh
- ops/lib/scripts/skill.sh

3.3 Scope and Safety

Objective: Fix linter path resolution to green repo-gates; move SKILL.md to library; remove redundant documentation.

Target Files:

- tools/lint/context.sh
- tools/lint/truth.sh
- tools/lint/library.sh
- tools/verify.sh
- SKILL.md (move to docs/library/SKILLS.md)
- ops/lib/scripts/skill.sh (update paths)
- llms.txt (update paths)
- ops/lib/project/user-STELA.md (delete)
- ops/lib/project/INTEGRATION.md (delete)
- docs/library/INTEGRATION.md (delete)
- docs/library/INDEX.md (update registry)

3.4 Execution Plan

1. **Refactor Linters:** Update all tools/ scripts to use REPO_ROOT="$(git rev-parse --show-toplevel)" for path resolution.
2. **Relocate SKILL.md:** Move SKILL.md to docs/library/SKILLS.md.
3. **Update Skill Script:** Modify ops/lib/scripts/skill.sh to point SKILL_FILE to the new

location.

4. **Prune Docs:** Delete ops/lib/project/user-STELA.md, ops/lib/project/INTEGRATION.md, and docs/library/INTEGRATION.md.
5. **Update Registry:** Update docs/library/INDEX.md to reflect the move of SKILL.md and removal of INTEGRATION.md.

1

Works cited

1. dump-full-main-f2258f404.manifest.txt
2. Manipulate folder path in shell script variable : r/bash - Reddit, accessed February 4, 2026, https://www.reddit.com/r/bash/comments/1nbp7ve/manipulate_folder_path_in_shell_script_variable/
3. DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)" How does that work?, accessed February 4, 2026, https://stackoverflow.com/questions/39340169/dir-cd-dirname-bash-source0-pwd-how-does-that-work
4. Agentic Software Engineering: Foundational Pillars and a Research Roadmap - arXiv, accessed February 4, 2026, https://arxiv.org/html/2509.06216v2
5. Bash: From within a script, refer to the script's parent directory | by Forest Dewberry | Medium, accessed February 4, 2026, https://medium.com/@forest.dewberry/bash-from-within-a-script-refer-to-the-scripts-parent-directory-2eb10fab5b13
6. Refer to a file under the same directory of a script found in $PATH, accessed February 4, 2026, https://unix.stackexchange.com/questions/17717/refer-to-a-file-under-the-same-directory-of-a-script-found-in-path
7. Why AI Agents are Good Software - Devansh - Medium, accessed February 4, 2026, https://machine-learning-made-simple.medium.com/why-ai-agents-are-good-software-0fc97b7a4d25
8. Best practices for repository structure : r/git - Reddit, accessed February 4, 2026, https://www.reddit.com/r/git/comments/dp5w0m/best_practices_for_repository_structure/
9. A Practical Guide to Agentic Software Development, accessed February 4, 2026, https://sevenpeakssoftware.com/blog/a-practical-guide-to-agentic-software-development